VIET NAM NATIONAL UNIVERSITY HO CHI MINH

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

Final Project

Data Structure Visualizer

Course: Data Structures - CS163

Student list:

Nguyễn Đình Minh Huy (25125083)

Nguyễn Đức Huy (25125014)

Trần Như Khải (25125045)

Nguyễn Anh Kiệt (25125074)

Group 09 - Class 25A01

28th April 2026

Contents

1 Abstract **1**

2 Introduction **1**

2.1 Key Features 1

2.2 Supported Data Structures and Algorithms 2

2.3 Technical Architecture 3

3 Task Assignment **3**

4 Data Storage **6**

4.1 Data Life-cycle and Storage Architecture 6

4.1.1 Transient Input Storage 6

4.1.2 Persistent Structural Representation 6

5 Architecture **7**

5.1 Architectural Overview: MVC Intuition 7

5.2 Detailed System Design and Interfaces 8

5.2.1 The `IVisualizable` Abstraction 8

5.2.2 Timeline and Frames 9

5.3 Visitor Design Pattern for Rendering 10

5.3.1 Pattern Implementation 10

6 Implementation **10**

6.1 Controller Layer Implementation 11

6.1.1 `AppEngine` (Application Orchestrator) 11

6.1.2 `Playback` (Timeline Management) 12

6.1.3 `DataManager` (Data Ingestion Logic) 12

6.2 Model Layer Implementation 13

6.2.1 Singly Linked List 13

6.2.2 Graph Structures (Adjacency List & Adjacency Matrix) 13

6.2.3 Grid Graph (Pathfinding Grids) 14

6.2.4 Binary Heap 14
6.2.5 AVL Tree (Self-Balancing BST) 15
6.3 View Layer Implementation 15
6.3.1 `UIManager` (The Mediator) 15
6.3.2 `Renderer` (The Visitor Pattern) 16
6.3.3 Dynamic Theming and Custom Animations 16
6.4 Animation System 17
6.4.1 Mathematical Foundations 17
6.4.2 Data Structure Animations 18
6.4.3 UI Animations 19
6.4.4 Main Menu Cinematic Decorations 20

7 Technical Problems **22**
7.1 Input Event Overlap (SFML vs. ImGui) 22
7.2 Playback State Desynchronization 22
7.3 Refactoring IDataStructure into IVisualizable and Sub-interfaces 22
7.4 State Encapsulation Leaks 23
7.5 Integer Overflow and Memory Safety 23
7.6 ImGui-SFML Lifecycle Assertions 24
7.7 Visualizing Array-Backed Heaps as Trees 24
7.8 Invalid Formats in File Input 24
7.9 Tracking State in Recursive Algorithms 24
7.10 Asset Path Resolution in Production 25
7.11 Circular Dependencies in Frame Payload 25
7.12 UI Component Hardcoding and Consistency 25
7.13 Payload Type-Checking Overhead 25

8 Features Demo **26**
8.1 System Overview 26
8.2 Core Features 26
8.3 Demonstration Scenarios 26
8.4 Discussion 27

9 Conclusion **27**

9.1 Summary of Achievements 27

9.2 Limitations 27

9.3 Future Work 28

References **29**

1 Abstract

Understanding abstract data structures is a fundamental challenge for computer science students due to the lack of intuitive visualization. This project presents an interactive data structure visualization application developed in C++ to address this limitation.

The system supports real-time visualization of core data structures, including Singly Linked Lists, Heaps, AVL Trees, and Graphs with shortest path algorithms. It provides step-by-step animation of operations such as insertion, deletion, and traversal, enabling users to observe dynamic changes during execution.

To ensure smooth and responsive interaction, the system leverages graphical libraries and adopts a modular architecture that synchronizes algorithm execution with user interface rendering.

Overall, the application enhances conceptual understanding and demonstrates the effectiveness of interactive visualization as a learning tool for data structures and algorithms.

2 Introduction

Learning Data Structures and Algorithms (DSA) is a fundamental stepping stone for computer science students and software engineers. However, the abstract nature of these concepts—such as tree balancing rotations, dynamic graph traversals, and heap properties—often poses a significant challenge. Traditional learning methods relying on static textbook diagrams or raw console outputs frequently fail to capture the dynamic, step-by-step transformations that occur in memory during algorithm execution.

To bridge this educational gap, the **Data Structure Visualizer** was developed. It is a high-performance, interactive, and visually engaging desktop application designed to bring abstract algorithms to life. Built using modern C++, the Simple and Fast Multimedia Library (SFML), and Dear ImGui, this application transcends standard educational tools by providing a seamless, game-like user experience.

2.1 Key Features

The application stands out by offering a comprehensive suite of interactive features designed to enhance cognitive understanding:

- **Timeline Playback System:** Users can play, pause, step forward, or step backward through an algorithm's execution. Operating like a "movie projector" for data structures, it allows users to deeply analyze complex steps at their own pace, with fully adjustable execution speeds.
- **Real-time Pseudocode Highlighting:** As the visualizer animates a specific step, the corresponding line of pseudocode is highlighted in a dedicated Code Panel. This creates a direct cognitive link between abstract code and visual behavior.
- **Cinematic UI/UX and Animations:** Moving away from rigid, snapping graphics, the application utilizes linear interpolation (Lerp), spring physics, and color blending. Every node movement, tree rotation, and state transition is rendered with extreme smoothness to prevent visual confusion.
- **Dynamic Theming:** Features a meticulously designed interface with full support for Light Mode and Dark Mode. The user interface integrates Cyberpunk and Holographic aesthetics to keep the learning process visually stimulating.
- **Flexible Data Input:** Users can initialize and modify structures via manual keyboard input, randomized generation, or by importing external datasets from files.

2.2 Supported Data Structures and Algorithms

The visualizer currently supports a robust library of standard and advanced data structures:

- **Linear Structures:** Singly Linked List (Insert, Delete, Search, Update).
- **Trees & Heaps:** AVL Tree (featuring automatic self-balancing rotations), Min Heap, and Max Heap.
- **Grid Environments:** Support for maze generation, obstacle placement, Breadth-First Search (BFS), and the A* Pathfinding algorithm.
- **Node-Edge Graphs:** Directed and Undirected graphs represented via Adjacency Matrices or Adjacency Lists. Supported algorithms include Dijkstra, Bellman-Ford, Floyd-Warshall, Johnson's Algorithm, and A*.

2.3 Technical Architecture

Under the hood, the application follows a strict *Model-View-Controller (MVC)* architecture.

- **The Model** handles the mathematical data representations and generates a timeline of discrete *Frames*.
- **The View** is powered by SFML for high-framerate 2D hardware-accelerated rendering and Dear ImGui for creating flat-design, highly responsive control panels.
- **The Controller** acts as the central engine (*AppEngine*), bridging user inputs, playback states, and continuous visual updates.

By combining robust algorithmic logic with modern visual aesthetics, the Data Structure Visualizer serves as both an effective self-study companion and a powerful demonstration tool for computer science education.

3 Task Assignment

Below is the detailed task assignment for each member of the group.

Student ID	**Name**	**Task**
		Continued on next page

Table 1 – continued from previous page

Student ID	Name	Task

Table 1 – continued from previous page

Student ID	Name	Task

Table 1 – continued from previous page

Student ID	Name	Task

4 Data Storage

4.1 Data Life-cycle and Storage Architecture

The system distinguishes between two primary forms of data: *Transient Input Data* and *Structural State Data*. This separation ensures that the raw input remains immutable while the visualization undergoes complex transformations.

4.1.1 Transient Input Storage

Before a data structure is constructed, raw values are stored in centralized buffers managed by the `DataManager`.

- **Schema Consistency**: Whether the data originates from a `.txt` file, a random generator, or manual console input, it is normalized into standard containers (`std::vector`).
- **Buffer Management**: The system follows a "Clear-on-Load" strategy. Each new ingestion command triggers a memory reset of the transient buffers to prevent data leakage between unrelated visualization sessions.

4.1.2 Persistent Structural Representation

Once the "Build" command is issued, the Model layer (e.g., `AVLTree`, `Graph`) takes ownership of the transient data and converts it into a persistent representation:

- **Node-Link Persistence**: Trees and Graphs are stored as pointer-based structures or adjacency lists, where each node maintains its own state (value, color, position).
- **Historical State Buffer**: As the algorithm progresses, the system does not overwrite the current state but stores "Snapshots" (Payloads) in the `Timeline`. This allows the storage to represent the data not just in its current form, but across its entire execution history.

5 Architecture

5.1 Architectural Overview: MVC Intuition

The application strictly adheres to the Model-View-Controller (MVC) design pattern. This architectural choice separates the internal representations of data structures, the logic that manipulates them, and the graphical interfaces presented to the user. This separation of concerns allows for a highly modular, maintainable, and extensible codebase.
The architecture is divided into three interconnected categories of components:

- **Controller**: Orchestrates the application. The central `AppEngine` class serves as the primary coordinator, processing user inputs and coordinating changes across the system. It delegates playback controls to the `Playback` controller, which manages the flow of time for step-by-step animations.
- **Model**: Represents the mathematical data structures, their algorithms, and their visual states over time. The `IVisualizable` interface acts as the base abstraction for all concrete implementations (such as Linked Lists, Graphs, and Heaps). When an algorithm is executed, the structure generates a `Timeline` composed of discrete `Frame`s. Each frame captures the exact state of the structure at a given execution step.
- **View**: Responsible for rendering the visual representation of the `Model`'s state. The `UIManager` handles modern graphical user interface elements via Dear ImGui, providing interactive menus and code panels. The `Renderer` utilizes SFML to draw graphical shapes based on the data contained in the current `Frame`.

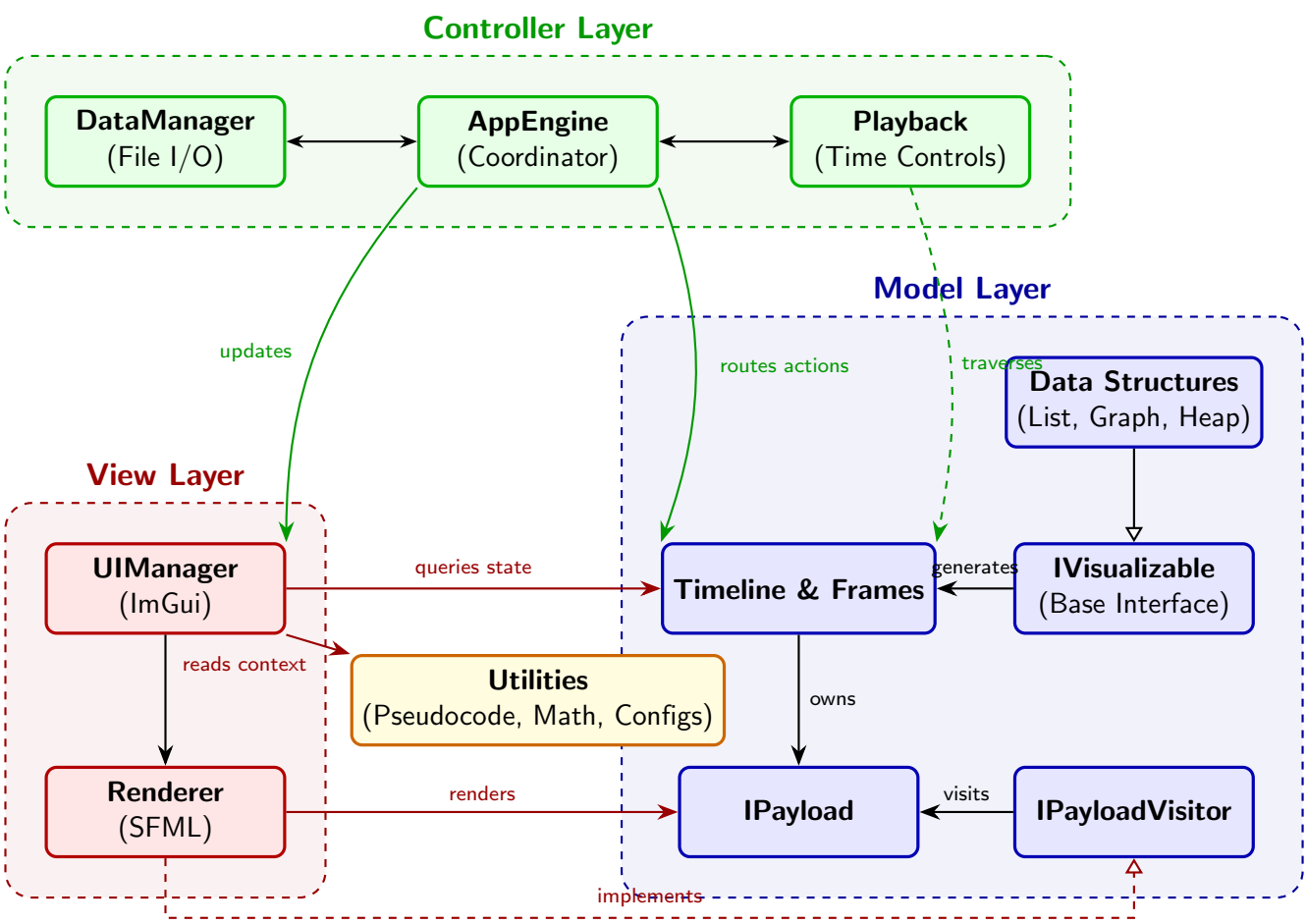

Figure 1: Application Architecture Diagram detailing the relationships between MVC components.

5.2 Detailed System Design and Interfaces

A cornerstone of the application's design is its extensive use of abstract interfaces. This approach enforces loose coupling, allowing the front-end rendering logic to remain agnostic to the internal implementations of the back-end data structures.

5.2.1 The `IVisualizable` Abstraction

Every data structure in the application inherits from the `IVisualizable` interface. This ensures that the `AppEngine` can interact with an AVL Tree, a Graph, or a Singly Linked List using a unified API. When a user action is dispatched (e.g., inserting a node), the specific data structure executes its algorithm and generates a `Timeline`.

5.2.2 Timeline and Frames

Instead of immediately drawing the results of an algorithm, the models generate discrete states.

- **Timeline**: A collection of `Frame` objects, representing the complete history of an algorithm's execution from start to finish.
- **Frame**: Captures a static snapshot of the visualization at a specific moment in time. A `Frame` holds the highlighted pseudocode line, variable states, and a list of graphical `IPayload` objects that need to be drawn.

This design effectively isolates the algorithmic logic from the real-time rendering loop. It natively supports playback features such as pausing, rewinding, and fast-forwarding, simply by selecting a different `Frame` within the `Timeline`.

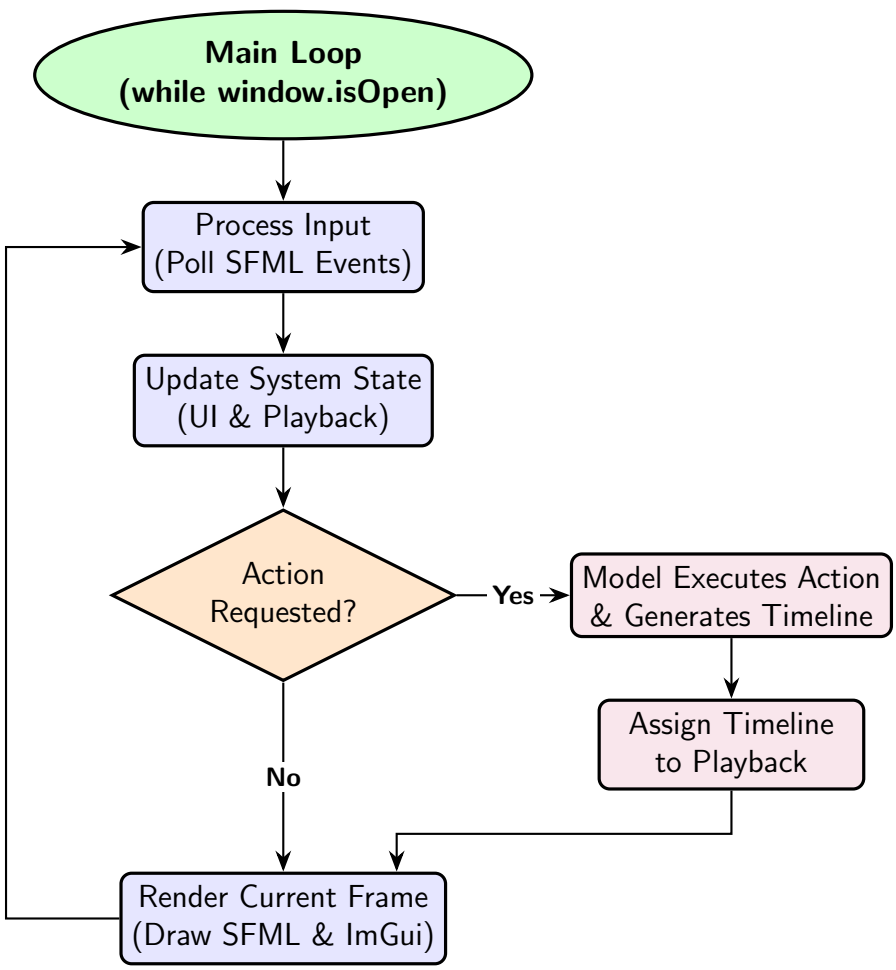

Figure 2: Application Execution Flow Loop.

5.3 Visitor Design Pattern for Rendering

A major challenge in a visualization system is handling the heterogeneous types of data that need to be drawn. A Singly Linked List requires rendering distinct nodes and directional arrows, while an Adjacency Matrix necessitates rendering a grid. If the `Renderer` were to use a long series of `if-else` statements to determine the type of an object before drawing it, the code would quickly become unmaintainable. Furthermore, giving the `Model` the responsibility to render itself would violate the MVC principles.

To solve this problem cleanly, the application employs the **Visitor Design Pattern**.

5.3.1 Pattern Implementation

The pattern relies on two primary abstractions:

1. `IPayloadVisitor`: An interface declaring a `visit()` method for every possible concrete payload type (e.g., `visit(const LinkedListPayload&)`, `visit(const GraphPayload&)`). The `Renderer` implements this interface.

2. `IPayload`: An interface representing a drawing instruction generated by the Model. It requires an `accept(IPayloadVisitor& visitor)` method.

When a `Frame` is rendered, the `Renderer` iterates through the `Frame`'s payload list and calls `payload->accept(*this)`. Through dynamic dispatch, the concrete payload's `accept` method is invoked. This method then immediately calls `visitor.visit(*this)`, effectively passing its strongly-typed self back to the `Renderer`.

This architecture guarantees that the `Model` never references SFML or rendering specifics, while the `Renderer` avoids costly runtime type checks (`dynamic_cast`), resulting in safe, robust, and highly decoupled code.

6 Implementation

This section details the concrete implementation of the core components within the system architecture. It outlines the specific mechanisms used in the Controller and Model layers, focusing on application orchestration, playback state management, and the underlying data structures utilized for visualizations.

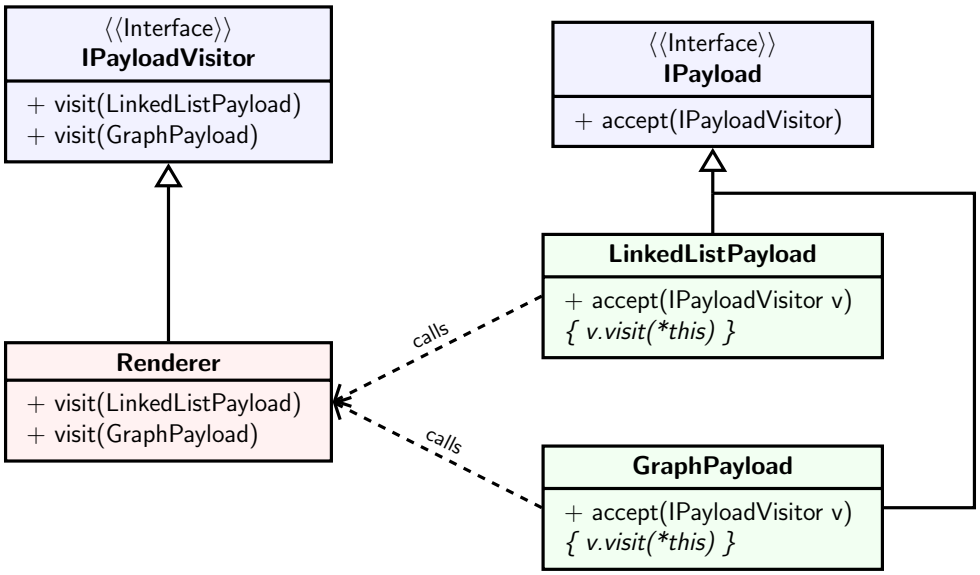

Figure 3: UML Class Diagram illustrating the Visitor Pattern for Payload rendering.

6.1 Controller Layer Implementation

The Controller layer acts as the nervous system of the application, bridging user interactions with underlying structural logic and managing the flow of time.

6.1.1 `AppEngine` (Application Orchestrator)

The `AppEngine` class encapsulates the primary SFML event loop and coordinates all subsystems. Its implementation handles several critical responsibilities:

- **Input Event Processing**: The `AppEngine` captures standard SFML events. If an event is not consumed by the `UIManager` (e.g., Dear ImGui captures a click on a menu button), it falls back to the `Renderer`. This allows seamless interaction, such as dragging and dropping graph nodes directly on the canvas without triggering UI elements.
- **View Manipulation (Zoom and Pan)**: To handle large data structures, `AppEngine` implements an interactive 2D camera system. Mouse scroll wheel events dynamically scale the `sf::View`, adjusting the zoom factor with clamped boundaries to prevent extreme scaling. Panning is implemented by tracking the mouse delta between `MouseButtonPressed` and `MouseMoved` events and shifting the `sf::View` center accordingly.
- **Data Action Routing**: When a user requests an algorithm (e.g., Insert, Delete, Run A*),

`AppEngine` queries the `UIManager` for the input parameters. It then resolves the required action to a specific `AlgorithmType`, retrieves the corresponding pseudocode via `PseudocodeManager`, and dispatches the command to the active `IVisualizable` instance. Once the Model generates a `Timeline`, it is immediately assigned to the `Playback` system.

6.1.2 `Playback` (Timeline Management)

The `Playback` system provides VCR-like controls (Play, Pause, Step Forward, Step Backward) over the generated visualization states. Instead of calculating animations on the fly, the `Playback` class acts as an iterator over a fully populated `Timeline`. Its `update(float deltaTime)` method tracks an internal `timeAccumulator`. When the accumulator exceeds the `baseStepDuration` (modified by a `speedMultiplier`), the system advances to the next `Frame` in the timeline. This design guarantees deterministic animation and allows users to jump instantly to any step in the algorithm's execution without recalculating the model's state.

6.1.3 `DataManager` (Data Ingestion Logic)

The `DataManager` class acts as the system's primary gateway for data, responsible for parsing raw inputs into structured formats that the Model layer can consume. Its implementation focuses on flexibility across multiple input modalities.

- **Input Parsing and Tokenization**: For console and file inputs, `DataManager` implements robust string parsing logic. Methods such as `inputFromConsole()` utilize `std::stringstream` to tokenize space-separated or comma-separated values, converting them into `std::vector<int>` or `Edge` objects while performing basic validation (e.g., ignoring non-numeric characters).

- **Randomization Engines**: The class provides a suite of randomizers (e.g., `randomDataPlanarGraph`, `randomDataDAG`). These are implemented using the C++ `<random>` library to ensure uniform distribution within user-defined constraints (min/max values, node density). For planar graphs, it incorporates coordinate generation logic to prevent nodes from overlapping during the initial layout phase.

- **Data Mapping and Distribution**: `DataManager` maintains specialized buffers for different structure types:

  - `std::vector<int>` for linear structures (Heaps, Arrays) and trees.
  - `std::vector<Edge>` for general graphs, storing source, destination, and weights.
  - `std::vector<std::vector<int»` for grid-based pathfinding algorithms.

- **Encapsulated Output**: To facilitate debugging and data persistence, the class implements `outputToFile()` and `outputToConsole()`. These methods serialize the current state of the input buffers back into human-readable formats, allowing users to save generated test cases for future sessions.

6.2 Model Layer Implementation

The Model layer houses the mathematical logic and the data payload generation mechanisms for the visualizer.

6.2.1 Singly Linked List

The `SinglyLinkedList` class implements the `IStandardStructure` interface. Internally, it manages a standard chain of `Node` pointers.

- **Payload Generation**: Every modification or traversal step in an algorithm (like insertion or searching) records a `Frame`. The structure captures its current state into a `std::vector<int>` and creates a `LinkedListPayload`. It also attaches an array of highlighted indices.

- **Algorithm Tracking**: For instance, during a search operation, the list iterates through its nodes. At each node, a `Frame` is generated with the current node's index highlighted and a descriptive message (e.g., "Comparing with node value X"). This provides the View with the exact context needed to highlight the corresponding visual node on the screen.

6.2.2 Graph Structures (Adjacency List & Adjacency Matrix)

Graphs are implemented using the `IGraphStructure` abstraction, with concrete variants for Adjacency Lists (optimizing sparse graphs) and Adjacency Matrices (optimizing dense graphs).

- **Edge and Vertex Management**: Both variants support adding/removing vertices and directed/undirected edges with weights.

- **Pathfinding Algorithms**: Implementations for algorithms such as Dijkstra, Bellman-Ford, and A* are provided. During execution, the graph maps its internal state (visited nodes, current paths, edge weights) into a generic `GraphPayload`. Nodes and edges that are actively being evaluated are tagged with highlight flags within the payload, allowing the `Renderer` to change their colors dynamically.

6.2.3 Grid Graph (Pathfinding Grids)

The `GridGraph` structure simulates shortest-path algorithms (BFS, A*) on a 2D matrix, treating each cell as a node with implicit edges to its orthogonal and diagonal neighbors.

- **State Mapping**: The grid is represented by an array of `CellState` enums (Empty, Wall, Start, End, Path, Visited).
- **Algorithm Visualization**: When running A*, the algorithm maintains open and closed sets. At each expansion step, a `GridPayload` is generated. Cells in the open set are marked as 'Discovered', and the current shortest path is traced backward from the current node to the start node, marked as 'Path'. This allows users to visually comprehend the expanding frontier of the heuristic search.

6.2.4 Binary Heap

The `BinaryHeap` implementation provides a visualization of priority queue operations, utilizing a hybrid representation that bridges a logical complete binary tree with a physical contiguous array.

- **Internal Representation**: The structure is managed via `std::vector<int> heapArray`. The parent-child relationships are calculated using zero-based indexing arithmetic: a node at index i has children at $2i + 1$ and $2i + 2$, while its parent resides at $\lfloor (i - 1)/2 \rfloor$.
- **Heapify Mechanics**: The core logic is encapsulated in `shiftUp` and `heapify` (shift-down) methods. During an `insert` or `remove` operation, every swap between elements triggers the generation of a new `Frame`. This allows the `Renderer` to animate the "bubbling" effect of elements moving through the levels of the tree.
- **Polymorphic Comparison**: Through the `IHeapStructure` abstraction, the system supports both `MinHeap` and `MaxHeap`. The specific heap property is enforced by overriding the pure

virtual `compare()` method, which determines the priority during the restoration of the heap invariant.

6.2.5 AVL Tree (Self-Balancing BST)

The `AVLTree` implementation focuses on demonstrating the maintenance of the height-balance property ($|balance_factor| \leq 1$) through recursive rotations.

- **Recursive Balancing**: The structure uses a nested `Node` structure containing `height` and pointer-based child references. Following every `insert` or `remove` operation, the algorithm recursively traverses back to the root, updating heights and invoking the `balance()` method.
- **Rotation Visualization**: The four types of rotations (Left, Right, Left-Right, and Right-Left) are decomposed into discrete steps. When a rotation occurs, the `AVLTree` generates a `TreePayload` that updates the parent-child pointers and node positions. The `Timeline` captures the transient states where the balance factor is violated, highlighting the specific nodes that trigger the rotation logic.
- **Payload Mapping**: To facilitate rendering, the tree structure is flattened into a `TreePayload` containing `TreeNodeData` objects. Each node is assigned a unique ID and its current height, which the `Renderer` uses to compute the geometric layout on the 2D canvas, ensuring that subtrees do not overlap visually.

6.3 View Layer Implementation

The View layer is responsible for the visual representation of data structures and the management of user interactions. It adopts a dual-platform approach: utilizing the Simple and Fast Multimedia Library (SFML) for high-performance 2D canvas rendering, and Dear ImGui for immediate-mode overlay interfaces.

6.3.1 `UIManager` (The Mediator)

To prevent tight coupling between the graphical interface and the core algorithmic logic, the system employs the `UIManager` class acting as a Mediator.

- **Centralized UI Aggregation**: Instead of exposing individual UI components to the engine, `UIManager` encapsulates the `NavigationMenu`, `InputMenu`, `CodePanel`, and all playback widgets (`Button`, `Slider`).

- **State Synchronization**: The interface does not process logic; it merely polls user commands (e.g., `getInputAction()`, `getInputString1()`) and relays them to the `AppEngine`. Conversely, the engine updates the UI via the `syncPlaybackUI()` method, ensuring that buttons correctly reflect the current timeline state (e.g., disabling the 'Step Forward' button when the algorithm terminates).

6.3.2 `Renderer` (The Visitor Pattern)

The `Renderer` class is dedicated to drawing the algorithmic payloads onto the SFML canvas. It explicitly implements the `IPayloadVisitor` interface, leveraging the Visitor design pattern.

- **Polymorphic Rendering**: When the core generates a visual state, it passes a generic payload to the `Renderer`. Through overloaded `visit()` methods, the `Renderer` dynamically resolves the payload type (e.g., `TreePayload`, `GraphPayload`) and invokes the appropriate drawing subroutines.

- **Geometric Calculations**: The renderer abstracts complex SFML draw calls into reusable functions like `drawImageNode()` and `drawLineWithArrow()`, mathematically calculating boundary points based on the shape types (Circle or Rectangle) to ensure arrows precisely touch the edges of the nodes without overlapping.

6.3.3 Dynamic Theming and Custom Animations

To enhance the user experience, the system avoids static styling by implementing a global `Theme` singleton.

- **On-the-fly Customization**: The `Theme` structure holds global configurations for colors, typography, and asset paths, enabling seamless toggling between Dark and Light modes without requiring an application restart.

- **Advanced ImGui Widgets**: Moving beyond default ImGui components, the `UIanimation` namespace introduces custom, mathematically interpolated widgets. Functions such as `JuicySliderFloa`

and `CyberButton()` utilize linear interpolation (Lerp) to create smooth scaling, pulsing glows, and dynamic color transitions, providing a highly polished, modern interface.

6.4 Animation System

The animation system in the project is designed to be completely frame-independent, relying heavily on the `DeltaTime` (Δt) parameter. Instead of abruptly changing states, all transitions regarding coordinates, scale, and colors are computed via continuous mathematical functions, yielding a highly fluid visual experience (Game Feel).
The system is divided into four main components: Mathematical Foundations, Data Structure Animations, UI Animations, and Cinematic Menu Decorations.

6.4.1 Mathematical Foundations

Non-linear Interpolation (Exponential Decay Lerp) To ensure objects move smoothly to their targets without stuttering upon sudden direction changes, the project employs an interpolation formula with an exponentially decaying weight rather than a traditional linear Lerp:

$$V_{current} = V_{current} + (V_{target} - V_{current}) \times \left(1 - e^{-k \cdot \Delta t}\right) \tag{1}$$

Where:

- $V_{current}$: The current value (position, scale, RGB color).
- V_{target}: The target value.
- k: The convergence rate constant (e.g., $k = 15.0$ for translation, $k = 10.0$ for color transition).
- Δt: The elapsed time between two frames.

Sinusoidal Waves for Continuous Motion Continuous oscillating effects, such as pulsing glows, floating motions, and glitch noises, are driven by trigonometric functions based on real-time (`appTime`):

$$y = A \cdot \sin(\omega \cdot t + \phi) \tag{2}$$

Where A is the amplitude, ω is the angular frequency, and ϕ is the initial phase (often dependent on the object's `index` to create a cascading delay effect).

6.4.2 Data Structure Animations

The rendering process does not fetch mathematical coordinates directly. Instead, each node is managed by an isolated state structure `NodeAnimState`:

```
struct NodeAnimState {
    sf::Vector2f pos;
    float scale = 0.0f;
    float highlightAlpha = 0.0f;
    sf::Vector2f velPos = {0.0f, 0.0f}; // For Graph physics
    float velScale = 0.0f;
    float colorR = -1.0f, colorG = 0.0f, colorB = 0.0f;
};
```

Listing 1: Animation State Management Structure

Curved Paths in Trees and Heaps When elements in an AVL Tree or Heap swap positions, moving them in a straight line causes visual overlap and clutter. The project resolves this by adding a perpendicular vector to the trajectory when the distance is significant:

```
sf::Vector2f diff = targetPos - anim.pos;
if (std::abs(diff.x) > 1.0f || std::abs(diff.y) > 1.0f) {
    sf::Vector2f perp(-diff.y, diff.x); // Perpendicular vector
    anim.pos += (diff + perp * 0.3f) * lerpPos; // 0.3f is the curve factor
} else {
    anim.pos += diff * lerpPos; // Linear convergence when close
}
```

Listing 2: Curved Trajectory Algorithm

Spring-Damper Physics for Graphs For graph structures, nodes require a realistic bouncing effect upon appearance or interaction. A physical model of tension and damping is applied:

$$\vec{v}_{new} = \left(\vec{v}_{old} + (\vec{P}_{target} - \vec{P}_{current}) \cdot \text{tension} \cdot \Delta t\right) \cdot e^{-\text{damp} \cdot \Delta t} \tag{3}$$

```
float tension = 200.0f; float damp = 12.0f;
anim.velPos += (targetPos - anim.pos) * tension * currentDeltaTime;
anim.velPos *= std::exp(-damp * currentDeltaTime);
anim.pos += anim.velPos * currentDeltaTime;
```

6.4.3 UI Animations

The default ImGui interface is extremely static. To create a "Cyberpunk" aesthetic, the static library is overridden via the `UIanimation` namespace, tracking Hover and Active states independently using `ImGuiID`.

Particle System for Buttons When interacting with buttons (e.g., Play/Pause), a burst of light particles is emitted. The system manages particle lists tied to the button's ID. Upon clicking, the algorithm spawns $15 \sim 25$ particles radiating outward at random angles:

```
for (size_t i = 0; i < plist.size(); ) {
    plist[i].pos += plist[i].vel * dt; // Calculate velocity
    plist[i].vel.y += 200.0f * dt;     // Apply gravity
    plist[i].alpha -= dt * 1.5f;       // Fade out

    if (plist[i].alpha <= 0.0f) {
        plist.erase(plist.begin() + i); // Erase dead particles
    } else {
        // Render particle and glowing trail
        draw_list->AddCircleFilled(plist[i].pos, plist[i].size, pCol);
        draw_list->AddLine(plist[i].pos,
            plist[i].pos - ImVec2(plist[i].vel.x * 0.05f, plist[i].vel.y * 0.05f
    ),
            trailCol, plist[i].size);
        ++i;
    }
}
```

Listing 3: Particle Physics Update

Glitch Effect (RGB Split & Noise) For hacker-styled buttons (`Glitch Button`), the system overdraws the original text with Red and Cyan channels. They are offset by a displacement calcu-

lated from combined sine and cosine functions to simulate random noise:

```
float time = (float)ImGui::GetTime();
float noise = std::sin(time * 45.0f) * std::cos(time * 15.0f);
bool isGlitching = (hovered && noise > 0.5f);

if (isGlitching) {
    float offsetStr = 3.0f * noise;
    // Red channel skewed right
    draw_list->AddText(text_pos + ImVec2(offsetStr, 0), IM_COL32(255, 50, 50,
   200), label);
    // Cyan channel skewed left and down
    draw_list->AddText(text_pos + ImVec2(-offsetStr, offsetStr * 0.5f), IM_COL32
   (50, 255, 255, 200), label);
}
```

Interactive Juicy Slider The playback speed slider bypasses the internal ImGui behavior to calculate exact mouse coordinates, preventing crashes. Its animations include:

- **Track Expansion:** The thickness of the track increases upon hovering.
- **Shake Effect:** While dragging, the slider knob vibrates rapidly across the X and Y axes to simulate physical friction.
- **Text Bump:** Whenever the speed value changes, the text string pops up abruptly and smoothly settles down via the `bumpAnim` variable.

6.4.4 Main Menu Cinematic Decorations

The background of the Main Menu incorporates highly intricate decorative animations programmed directly in SFML to maximize user immersion. These are rendered within the `DecorationPayload` and `MenuAnimPayload`.

Interactive Hexagon Web & Matrix Rain A honeycomb grid covers the screen. The algorithm checks the Euclidean distance from each hexagon to the mouse cursor. If the distance is within the `lightRadius`, its Alpha multiplier increases, creating a "light chasing the mouse" effect.

$$\alpha_{mouse} = 1.0 - \left(\frac{\sqrt{dx^2 + dy^2}}{R_{light}} \right) \tag{4}$$

Simultaneously, Matrix data streams fall vertically, utilizing `std::fmod` to endlessly wrap coordinates around the screen dimensions.

Geometric Data Structure Emulations The renderer continuously draws conceptual representations of data structures in the background, complementing the visual theme:

- **Floating Binary Tree:** A binary tree sways back and forth using an `sf::Transform` matrix rotated by $\sin(\text{time} \cdot 0.5) \cdot 5.0°$.
- **Complete Graph (K_5):** 5 nodes are arranged in a circle using parametric equations $(R\cos\theta, R\sin\theta)$ and are fully interconnected, while the entire graph continuously rotates.
- **Wavy Linked List:** Nodes follow a horizontal sine wave path $\sin(\text{time} \cdot 3.0 + i \cdot 1.5)$. Small "data packets" travel along the edges between nodes, simulating data transfer.
- **3D Isometric Matrix:** A 2D grid of squares is projected into 3D isometric space by first scaling the Y-axis by 0.6 and then rotating the entire structure by 45°.
- **Sorting Histogram:** Bar heights fluctuate using overlapping sine and cosine waves, combined with a moving "scanline" highlight effect derived from modulo operations on the bar index.
- **Neural Network:** Connections between layers light up dynamically. The signal strength is calculated as $\sin(\text{time} \cdot 6.0 - \text{layer} \cdot 1.5 + n_1 \cdot 0.5 + n_2 \cdot 0.5)$. Edges with a signal > 0.85 are highlighted intensely.
- **Central Quantum Core:** Nested elliptical orbits rotate at different speeds. A central hexagon counter-rotates, surrounded by a pulsing holographic HUD and moving barcode data streams that mimic complex processing.

7 Technical Problems

7.1 Input Event Overlap (SFML vs. ImGui)

Problem: The system utilizes ImGui as an overlay on top of the SFML canvas. Initially, mouse click events intended for ImGui menus (such as clicking a text input field in the `InputMenu`) would "bleed through" to the underlying SFML layer. This caused unintended behaviors, such as accidentally triggering the Drag-and-Drop node functionality on the canvas while merely attempting to type a value.

Solution: We implemented an event-blocking mechanism within the `UIManager`. By wrapping ImGui's internal IO queries, we exposed methods such as `isMouseOverUI()` and `isKeyboardCapturedByUI()`. Before the `Renderer` or `AppEngine` processes any mouse press for canvas manipulation, they strictly poll these flags. If the mouse is actively hovering over an ImGui window, canvas interactions are safely ignored.

7.2 Playback State Desynchronization

Problem: The timeline navigation allows users to rapidly click "Step Forward" or manipulate the speed slider. However, spamming these controls could cause the UI state to desynchronize from the actual `Timeline` state managed by the `AppEngine`, occasionally leading to out-of-bounds frame access or conflicting button states (e.g., both Play and Pause appearing active simultaneously).

Solution: We enforced a strict unidirectional state flow. The `Playback` controller acts as the single source of truth for the active frame index. UI buttons no longer manipulate the timeline directly; instead, they send state-change requests to the `AppEngine`, which processes them sequentially. Boundary clamping was also implemented inside the `Timeline` class to prevent out-of-bounds indices.

7.3 Refactoring IDataStructure into IVisualizable and Sub-interfaces

Problem: Initially, the project relied on a single, monolithic `IDataStructure` interface. As the project scaled to support complex types like Graphs and Heaps alongside simple Arrays and Lists, this base interface became bloated. Standardizing functions across all data types forced structures to implement irrelevant methods (e.g., a Linked List having to implement graph-specific `addEdge`

methods), violating the Interface Segregation Principle.

Solution: We executed a major architectural refactoring by deprecating `IDataStructure` and introducing a lightweight base interface called `IVisualizable`. From this base, we branched out into three highly specialized sub-interfaces:

- `IStandardStructure`: For linear and basic structures (Lists, Arrays) requiring standard `insert`/`remove` operations.
- `IHeapStructure`: Dedicated to array-backed tree operations like `buildHeap()` and `heapify()`.
- `IGraphStructure`: Specifically tailored for handling `addVertex()`, `addEdge()`, and pathfinding matrices.

This separation of concerns drastically improved code maintainability and ensured that each data structure only implements the methods it strictly needs.

7.4 State Encapsulation Leaks

Problem: The `Frame` and `Timeline` classes originally exposed their internal states. This led to unintended side effects where external algorithmic functions could accidentally modify historical data.

Solution: We enforced strict encapsulation by moving internal states to private member variables and exposing only `const` getters and controlled setters, ensuring the integrity of the algorithm timelines.

7.5 Integer Overflow and Memory Safety

Problem: Using standard `int` types for array indices and graph sizes caused overflow risks and compiler compatibility warnings, particularly when interacting with the standard library's `std::vector`.

Solution: We conducted a codebase-wide refactoring to strictly enforce the use of `size_t` for all array loops, node IDs, and pointers. We also introduced designated sentinel values (e.g., `GlobalConst::INVALID_I` to represent null pointers safely.

7.6 ImGui-SFML Lifecycle Assertions

Problem: During the application exit sequence, the program occasionally crashed or threw assertions due to improper teardown orders between the SFML Window and ImGui contexts.

Solution: We restructured the `AppEngine` main loop and `UIManager` to guarantee that `ImGui::SFML::Shutdow` is called precisely after the final render cycle and before the SFML window is destroyed.

7.7 Visualizing Array-Backed Heaps as Trees

Problem: The Binary Heap data structure is backed by a 1D Array in memory, making it challenging to render dynamically as a 2D binary tree without nodes overlapping.

Solution: We developed a coordinate calculation algorithm using logarithmic math to recursively map 1D array indices ($2i + 1$ for left child, $2i + 2$ for right child) into visually appealing, non-overlapping 2D tree layouts.

7.8 Invalid Formats in File Input

Problem: Reading user-input or external files via `std::stringstream` was fragile. It would stall or crash when encountering garbage string values, negative numbers, or trailing spaces.

Solution: We implemented strong guard functions in the `DataManager`. By checking `fail()` flags and applying conditional parsing logic, the system now safely catches invalid inputs and returns graceful UI warnings instead of crashing.

7.9 Tracking State in Recursive Algorithms

Problem: Algorithms that rely heavily on recursion (such as AVL Tree balancing algorithms) made it exceptionally difficult to inject `timeline.addFrame()` snapshots without breaking the clean recursive flow or accumulating massive call stacks.

Solution: We explored passing state arguments down the recursion tree and extracting states via Breadth-First Search (BFS), ultimately finding a balance where rendering frames could be cleanly captured immediately after specific tree rotations.

7.10 Asset Path Resolution in Production

Problem: Assets (fonts, images) loaded via relative paths failed when the application was executed from different directory depths (e.g., inside an IDE vs. a deployed executable).

Solution: We engineered a robust asset root resolution function inside the `Theme` configuration that dynamically determines the runtime execution path and prepends it to all resource loading calls.

7.11 Circular Dependencies in Frame Payload

Problem: When expanding the `Frame` class to handle complex graphs, the `Edge` and `Node` structures were initially nested in ways that caused circular dependencies between the Model and rendering payloads.

Solution: We refactored the architecture to extract the `Edge` struct and other payload definitions outside of the `Frame` class. We adopted the Visitor Design Pattern for payload rendering, completely decoupling the Model state from the SFML drawing logic.

7.12 UI Component Hardcoding and Consistency

Problem: Early implementations of the UI components (Buttons, Sliders, Input Menus, Code Panels) relied heavily on hardcoded dimensions, positions, and colors. This made the UI fragile, visually inconsistent, and incredibly difficult to update across different views.

Solution: We executed a massive syntax refactoring across all view components, centralizing all stylistic configurations into a single `Theme` class. Components now query this single source of truth for colors and layout parameters, ensuring strict consistency and easy maintainability.

7.13 Payload Type-Checking Overhead

Problem: The initial rendering pipeline relied on extensive type-checking (e.g., using `dynamic_cast`) to determine whether a payload was an array, a graph, or a list before drawing it. This violated the Open-Closed Principle and resulted in messy, unmaintainable conditional logic in the `Renderer`.

Solution: We refactored the rendering system to implement the **Visitor Design Pattern**. The `Renderer` now acts as an `IPayloadVisitor`, allowing each payload type to natively accept the renderer and invoke its specific drawing function, eliminating the need for runtime type checks.

8 Features Demo

8.1 System Overview

The system is designed as an interactive visualization tool that integrates user interface controls, algorithm execution, and real-time graphical rendering. A demonstration video of the system is available at:

Demo Video: https://youtu.be/yeWcSl_vmm8?si=3cRV_46PZOY9nGyl

The video illustrates the core functionalities of the application, including user interaction, animation control, and visualization of multiple data structures.

8.2 Core Features

The system is organized into four main functional components:

Navigation and Theme Interface. The application provides a main menu for selecting different data structures. It also supports theme switching between light and dark modes, along with the ability to show or hide the pseudocode panel.

Control Panel. Users can input data through multiple methods, including random generation, console input, and file loading. The animation system includes controls such as play, pause, step forward, and step backward, as well as a speed adjustment slider ranging from 0.5x to 4.0x.

Pseudocode Tracking. The system highlights corresponding lines of pseudocode during execution, enabling users to directly map algorithm steps to visual changes on the screen.

Data Structure Operations Visualization. The application supports visualization for several data structures:

- Singly Linked List: insertion, deletion, and node updates.
- Heap: heapify operations during insertion and removal.
- AVL Tree: self-balancing through rotation operations.
- Graph: shortest path algorithms such as Dijkstra and Bellman-Ford.

8.3 Demonstration Scenarios

The demonstration video presents several typical use cases:

AVL Tree Balancing. When inserting nodes, the system automatically performs rotations to maintain balance. The animation highlights imbalance detection and the corresponding rotation steps.

Heap Operations. The heap structure is dynamically adjusted through heapify processes, allowing users to observe how elements are rearranged to maintain heap properties.

Graph Shortest Path Algorithms. Shortest path algorithms are visualized as propagation processes across the graph, where nodes and edges are progressively updated to reflect distance calculations.

8.4 Discussion

The integration of interactive controls, synchronized pseudocode, and animated visualization provides a comprehensive learning environment. The demonstration confirms that the system effectively bridges the gap between abstract algorithmic concepts and their practical execution.

9 Conclusion

9.1 Summary of Achievements

This project successfully developed an interactive data structure visualization system that demonstrates the fundamental operations of several core data structures. The implementation integrates algorithm execution, data management, and user interface components into a cohesive architecture. The system design separates responsibilities into distinct modules, including the algorithm engine, data management layer, and user interface controller, ensuring maintainability and scalability.

9.2 Limitations

Despite the achieved results, several limitations remain. First, developing a responsive and intuitive user interface using existing graphical libraries posed significant challenges. Additionally, maintaining synchronization between the user interface, data layer, and algorithm execution required careful system design.

From a functional perspective, the system currently provides limited direct interaction with data structures. Furthermore, performance and visualization clarity may degrade when handling large

datasets.

9.3 Future Work

Future improvements will focus on expanding the range of supported data structures, including B-Trees, Tries, and Hash Tables. Enhancing user interaction is also a priority, allowing operations to be performed through direct manipulation such as mouse actions and keyboard shortcuts.

In addition, further customization options for visualization themes and node representations will be explored, along with potential optimization for handling larger datasets more efficiently.

References

[1] Dan Bernstein. Hash functions (djb2) - comp.lang.c usenet, 1991. Accessed: 2025-12-28.

[2] Eraser.io. Eraser diagram tool. https://eraser.io/, 2025. Online tool used to design UML diagrams and system architecture illustrations.

[3] Google. Gemini. https://gemini.google.com/, 2025. Generative AI model used to support progress tracking, multithreading concepts, cross-platform implementation, code refactoring, and report writing.

[4] OpenAI. Chatgpt. https://chat.openai.com/, 2025. AI assistant used to help draft technical reports, convert content to LaTeX, and format tables, diagrams, and documentation based on student-provided input.